

Python - Join Tuples

Joining Tuples in Python

Joining **tuples** in Python refers to combining the elements of multiple tuples into a single **tuple**. This can be achieved using various methods, such as concatenation, list comprehension, or using built-in functions like `extend()` or `sum()`.

Joining tuples does not modify the original tuples but creates a new tuple containing the combined elements.

Joining Tuples Using Concatenation ("+") Operator

The concatenation operator in Python, denoted by `+`, is used to join two sequences, such as strings, lists, or tuples, into a single sequence. When applied to tuples, the concatenation operator joins the elements of the two (or more) tuples to create a new tuple containing all the elements from both tuples.

We can join tuples using the concatenation operator by simply using the `+` symbol to concatenate them.

Example

In the following example, we are concatenating the elements of two tuples "T1" and "T2", creating a new tuple "joined_tuple" containing all the elements from both tuples –

```
# Two tuples to be joined
T1 = (10,20,30,40)
T2 = ('one', 'two', 'three', 'four')
# Joining the tuples
joined_tuple = T1 + T2
```

```
# Printing the joined tuple
print("Joined Tuple:", joined_tuple)
```

Following is the output of the above code –

```
Joined Tuple: (10, 20, 30, 40, 'one', 'two', 'three', 'four')
```

Joining Tuples Using List Comprehension

List comprehension is a concise way to create lists in Python. It is used to generate new lists by applying an expression to each item in an existing iterable, such as a list, tuple, or range. The syntax for list comprehension is –

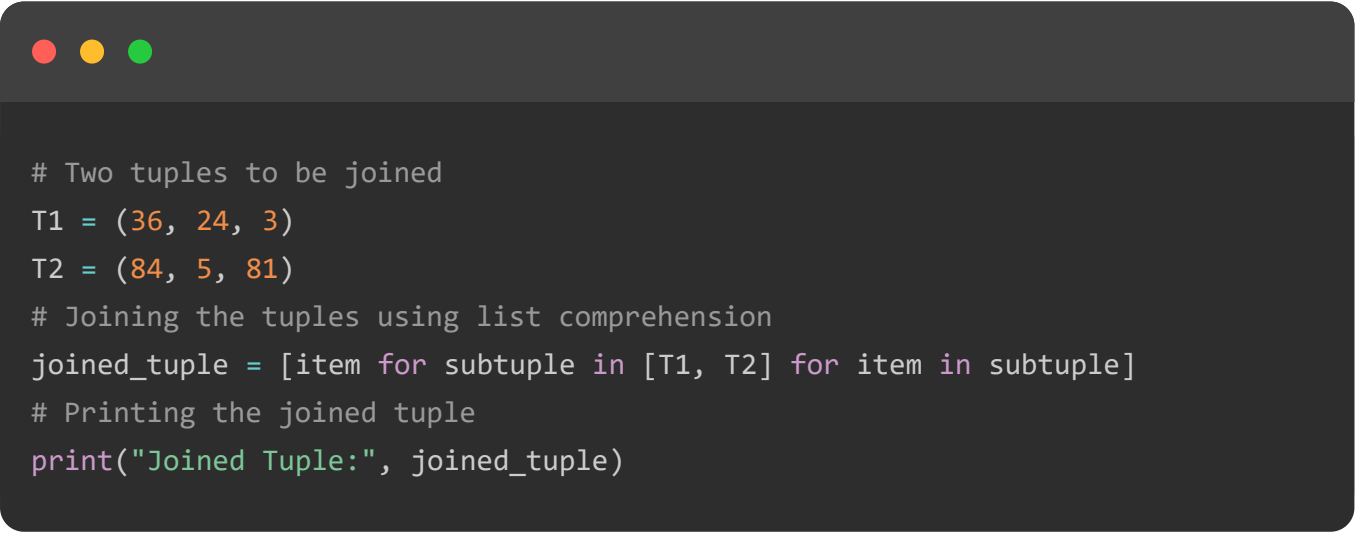
```
new_list = [expression for item in iterable]
```

This creates a new list where **expression** is evaluated for each **item** in the **iterable**.

We can join a tuple using list comprehension by iterating over multiple tuples and appending their elements to a new tuple.

Example

In this example, we are joining two tuples, T1 and T2, into a single tuple using list comprehension. The resulting tuple, `joined_tuple`, contains all elements from both T1 and T2 –



```
# Two tuples to be joined
T1 = (36, 24, 3)
T2 = (84, 5, 81)
# Joining the tuples using list comprehension
joined_tuple = [item for sublist in [T1, T2] for item in sublist]
# Printing the joined tuple
print("Joined Tuple:", joined_tuple)
```

Output of the above code is as follows –

Joined Tuple: [36, 24, 3, 84, 5, 81]

Joining Tuples Using extend() Function

The Python extend() function is used to append elements from an iterable (such as another list) to the end of the list. This function modifies the original list in place, adding the elements of the iterable to the end of the list.

The extend() function is not used for joining tuples in Python. It is used to extend a list by appending elements from another iterable (such as another list), effectively merging the two lists together.

We can join tuples using the extend() function by temporarily converting the tuples into lists, performing the joining operation as if they were lists, and then converting the resulting list back into a tuple.

Example

In the following example, we are extending the first tuple "T1" by converting it into a list "L1", then adding elements from the second tuple "T2" by first converting it into a list "L2", and finally converting the merged list back into a tuple, effectively joining the two tuples –

```
T1 = (10,20,30,40)
T2 = ('one', 'two', 'three', 'four')
L1 = list(T1)
L2 = list(T2)
L1.extend(L2)
T1 = tuple(L1)
print ("Joined Tuple:", T1)
```

The output obtained is as shown below –

Joined Tuple: (10, 20, 30, 40, 'one', 'two', 'three', 'four')

Join Tuples using sum() Function

In Python, the `sum()` function is used to add up all the elements in an iterable, such as a list, tuple, or set. It takes an iterable as its argument and returns the sum of all the elements in that iterable.

We can join a tuple using the `sum()` function by providing the tuple as an argument to the `sum()` function. However, since the `sum()` function is specifically designed for numeric data types, this method only works for tuples containing numeric elements. It will add up all the numeric elements in the tuple and return their sum.

Syntax

Following is the syntax for using the `sum()` function to join tuples in Python –

```
result_tuple = sum((tuple1, tuple2), ())
```

Here, the first argument is a tuple containing the tuples to be joined. The second argument is the starting value for the sum. Since we are joining tuples, we use an empty tuple `()` as the starting value.

Example

In this example, the elements of the first tuple are first appended to an empty tuple. Then elements from the second tuple are appended, resulting in a new tuple that is a concatenation of the two –

```
T1 = (10,20,30,40)
T2 = ('one', 'two', 'three', 'four')
T3 = sum((T1, T2), ())
print ("Joined Tuple:", T3)
```

After executing the above code, we get the following output –

```
Joined Tuple: (10, 20, 30, 40, 'one', 'two', 'three', 'four')
```

Joining Tuples using for Loop

A **for loop** in Python is used for iterating over a sequence (such as a list, tuple, string, or range) and executing a block of code for each element in the sequence. The loop continues until all elements have been processed.

We can join a tuple using a for loop by iterating over the elements of one tuple and appending each element to another tuple with the "+"=" operator.

Example

In the following example, we are iterating over each element in tuple T2, and for each element, we are appending it to tuple T1, effectively joining the two tuples –

```
T1 = (10,20,30,40)
T2 = ('one', 'two', 'three', 'four')
for t in T2:
    T1+=(t,)
print (T1)
```

We get the output as shown below –

```
(10, 20, 30, 40, 'one', 'two', 'three', 'four')
```

TOP TUTORIALS

- Python Tutorial
- Java Tutorial
- C++ Tutorial
- C Programming Tutorial
- C# Tutorial
- PHP Tutorial
- R Tutorial
- HTML Tutorial
- CSS Tutorial
- JavaScript Tutorial
- SQL Tutorial

TRENDING TECHNOLOGIES

- Cloud Computing Tutorial

[Amazon Web Services Tutorial](#)

[Microsoft Azure Tutorial](#)

[Git Tutorial](#)

[Ethical Hacking Tutorial](#)

[Docker Tutorial](#)

[Kubernetes Tutorial](#)

[DSA Tutorial](#)

[Spring Boot Tutorial](#)

[SDLC Tutorial](#)

[Unix Tutorial](#)

CERTIFICATIONS

[Business Analytics Certification](#)

[Java & Spring Boot Advanced Certification](#)

[Data Science Advanced Certification](#)

[Cloud Computing And DevOps](#)



Chapters ▾

Categories ≡

[DevOps Certification](#)

[Game Development Certification](#)

[Front-End Developer Certification](#)

[AWS Certification Training](#)

[Python Programming Certification](#)

COMPILERS & EDITORS

[Online Java Compiler](#)

[Online Python Compiler](#)

[Online Go Compiler](#)

[Online C Compiler](#)

[Online C++ Compiler](#)

[Online C# Compiler](#)

[Online PHP Compiler](#)

[Online MATLAB Compiler](#)

[Online Bash Terminal](#)

[Online SQL Compiler](#)

[Online Html Editor](#)

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.